Q1) You have to write a function called **alternateMaxMin()** array of integers of length **n**, you have to **modify the array** in such a way that the even indices contain **n/2** smallest values in ascending order and the odd indices contain the **n/2** largest values are in descending order. You must only use **Minheap** to solve this problem. The given **n** is always **even.**

Assume: The **MinHeap** class is already implemented with **insert**() and **extract**() methods along with the necessary helper methods. You do not need to write the **MinHeap** class. Simply create an object of the class and use it accordingly.
Note: You must utilize a **MinHeap** to solve the problem. You are not allowed to sort the array explicitly. Your solution time complexity should be less than $O(N^2)$. You are not allowed to use any additional data structures except the heap.

Sample Given Array	**Aftercallingalternate MaxMin()**
`arr = [5,7,1,3,4,10]`	`arr=[1,10,3,7,4,`

	5]
Explanation: Here, the length of the array,n=6. The smallest 3 integers are 1,3,4 which occupy the even indices = (0,2,4) in ascending order. The largest 3 integers 10,7,5 occupy the odd indices in descending order.	
arr = [5,4,3,-2,6,-4,9,1]	a r r = [- 4 , 9 , - 2 , 6 , 1 , 5 , 3 , 4]
Explanation: Here, the length of the array,n=8. The smallest 4 integers are -4,-2,1-3 which occupy the even indices = (0,2,4,6) in ascending order. The largest 4 integers 9,6,5,4 occupy the odd indices in descending order.	
PYTHON FUNCTION SIGNATURE	**J A V A M E T H O D S I G N A T U R E**
def alternateMaxMin(arr): *#todo*	*s t a t i c*

```
void alternateMaxMin(int[] arr){//todo}
```

Short Questions & MCQ

1. Which of the following statements accurately describes the defining property of a Min-Heap? [1]

- **A.** The left child of any node is always smaller than its right child.
- **B.** The left subtree contains smaller values and the right subtree contains larger values relative to the parent.
- **C.** Every node's value is less than or equal to the values of its children.
- **D.** It guarantees a worst-case time complexity for searching any arbitrary element.

2. Which type of traversal on a Binary Search Tree will visit the nodes in ascending sorted order? [1]

A. Pre-order traversal **B.**Level-order traversal **C.**In-order traversal **D.**Post-order traversal

3. If the Post-order traversal of a valid Binary Search Tree results in the sequence **[15, 30, 25, 60, 50, 40]**, what is the root node of this tree? [1]

A. 15 **B.**25 **C.**40 **D.**Cannot be determined.

4. Consider an initially empty **Binary Search Tree**.

I. Simulate the insertion of the following sequence: **[50, 30, 70, 20, 40, 60, 80, 35]**. **Only draw the resulting tree**. [2]

II. Draw the tree again after deleting the node **50** from your drawn BST. [1]